BC01-FM04 · séance 3 · mardi 10 novembre 2026, matin

La liste d'emprunts · fiche de B

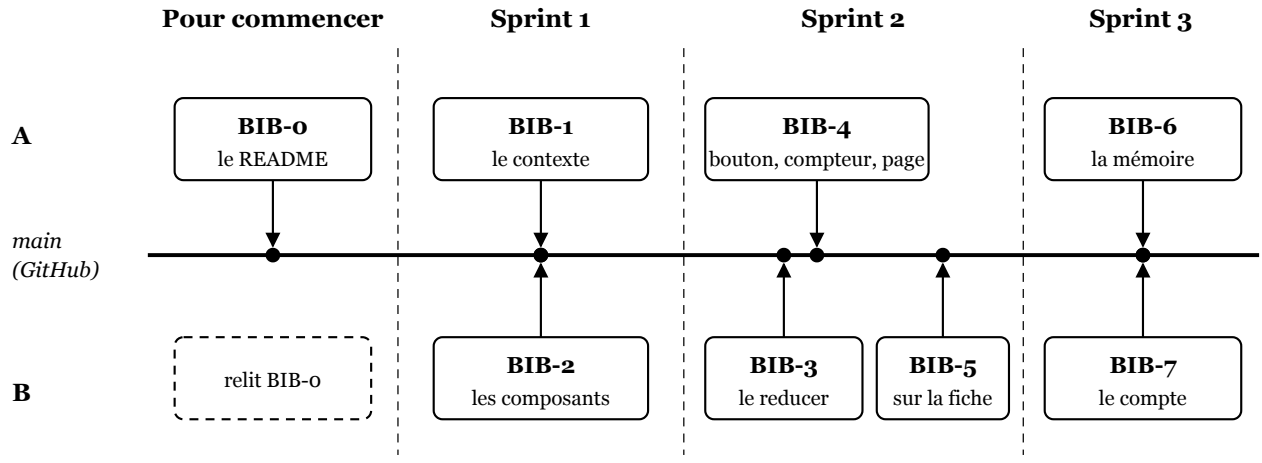
À rendre : `NOM1-NOM2-React-S3.zip`, le vendredi 13 novembre à 18 h 00
Avec : votre binôme, qui a la fiche A

Vous êtes deux développeurs. Le client veut une liste d'emprunts dans son catalogue. Le travail est découpé en tickets : chacun les siens, chacun sa branche.

Cette fiche dit **quoi faire**. Le **guide complet** explique **pourquoi** : ouvrez-le quand un morceau de code ou une commande vous échappe.

Vos tickets

Votre ticket	Ce que vous faites	A, en même temps
BIB-2	Les composants lisent la connexion	BIB-1 : le contexte de connexion
BIB-3	Le reducer et le contexte de la liste	BIB-4 : le bouton, le compteur, la page
BIB-5	Le bouton sur la fiche d'un livre	relit
BIB-7	Le compte affiche la liste	BIB-6 : la mémoire



Qui fait quoi, et quand. Chaque ticket rejoint main par une pull request, relue par l'autre.

Pour commencer : Git

1 · Votre poste

Dans le terminal de VS Code :

```
git --version
git config --global user.name "Prénom Nom"
git config --global user.email "vous@exemple.fr"
```

Pas de numéro de version ? Installez Git : **git-scm.com** sous Windows, `xcode-select --install` sous macOS. Il vous faut aussi un compte sur **github.com**.

---

2 · Le dépôt

Pendant que A crée le dépôt, lisez `README.md` et `TICKETS.md`, dans le zip.

Acceptez l'invitation de A (elle arrive par e-mail), puis :

```
git clone https://github.com/IDENTIFIANT-DE-A/catalogue-react.git
```

Vous devez voir : un nouveau dossier `catalogue-react/`, avec tout le projet de A.

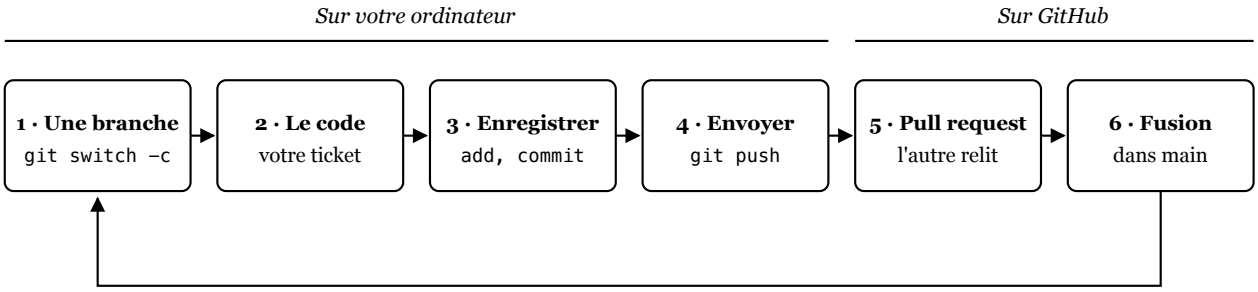
Ouvrez ce dossier dans VS Code, et lancez le projet :

```
npm install
npm run dev
```

Vous devez voir : le catalogue, à l'adresse http://localhost:5173.

3 · La boucle d'un ticket

La même pour tous vos tickets. Gardez cette page sous les yeux.



La boucle d'un ticket : trois gestes sur votre ordinateur, trois sur GitHub

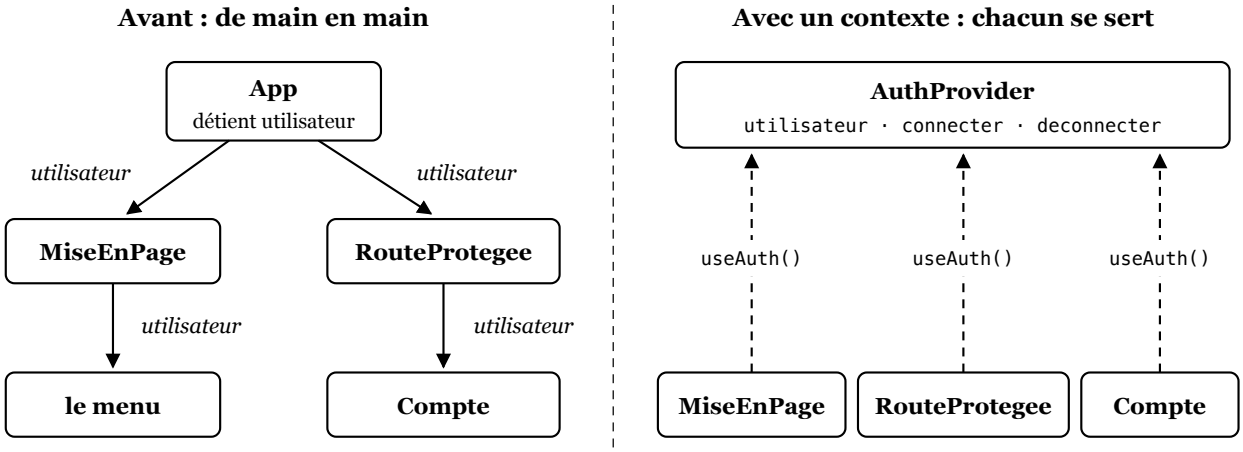
Quand	**Quoi**
Avant le ticket	`git switch main`, `git pull`, `git switch -c LA-BRANCHE`
Le ticket est fini	`git add .`, `git commit -m "LE MESSAGE"`, `git push -u origin LA-BRANCHE`
Sur GitHub	*Compare & pull request*, puis *Create pull request*
Votre binôme	lit *Files changed*, vous lui expliquez, il clique *Review changes → Approve*
Vous	*Merge pull request*, puis *Delete branch*
Tous les deux	`git switch main`, `git pull`

BIB-0 · Le premier aller-retour

A modifie `README.md`. Relisez sa pull request, et approuvez-la.

Vous devez voir : après `git pull`, votre `README.md` a changé, sans que vous y ayez touché.

Sprint 1 · La connexion



Ce que change le sprint 1 : l'utilisateur ne descend plus de main en main

Le contrat : `useAuth()` renvoie `{ utilisateur, connecter, deconnecter }`.

BIB-2 · Les composants lisent la connexion

Branche : `bib-2-lire-la-connexion`. Message : `BIB-2 : les composants lisent la connexion`.

Le fichier `src/contexts/AuthContext.jsx` est un bouchon : personne n'est connecté, c'est normal.

Remplacez tout le contenu de `src/App.jsx` par :

```
import { Routes, Route } from 'react-router';
import MiseEnPage from './components/MiseEnPage.jsx';
import RouteProtegee from './components/RouteProtegee.jsx';
import Accueil from './pages/Accueil.jsx';
import Catalogue from './pages/Catalogue.jsx';
import FicheLivre from './pages/FicheLivre.jsx';
import Connexion from './pages/Connexion.jsx';
import Compte from './pages/Compte.jsx';
import PageIntrouvable from './pages/PageIntrouvable.jsx';

// Les adresses de l'application : une route = une adresse = une page.
export default function App() {
  return (
    <Routes>
      <Route element={<MiseEnPage />}>
        <Route index element={<Accueil />} />
        <Route path="catalogue" element={<Catalogue />} />
        <Route path="livres/:id" element={<FicheLivre />} />
        <Route path="connexion" element={<Connexion />} />
        <Route
          path="compte"
          element={
            <RouteProtegee>
              <Compte />
            </RouteProtegee>
          }
        />
        <Route path="*" element={<PageIntrouvable />} />
      </Route>
    </Routes>
  );
}
```

La mise en page.

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { Link, NavLink, Outlet } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
export default function MiseEnPage({ utilisateur, onDeconnexion }) {
```

par :

```
export default function MiseEnPage() {
  const { utilisateur, deconnecter } = useAuth();
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
                <button type="button" onClick={onDeconnexion}>Se
déconnecter</button>
```

par :

```
                <button type="button" onClick={deconnecter}>Se déconnecter</button>
```

La garde.

Dans `src/components/RouteProtegee.jsx`, juste après la ligne `import { Navigate, useLocation } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/RouteProtegee.jsx`, remplacez :

```
export default function RouteProtegee({ utilisateur, children }) {
```

par :

```
export default function RouteProtegee({ children }) {
  const { utilisateur } = useAuth();
```

La connexion.

Dans `src/pages/Connexion.jsx`, juste après la ligne `import { adherents } from '../data/adherents.js';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
export default function Connexion({ onConnexion }) {
```

par :

```
export default function Connexion() {
  const { connecter } = useAuth();
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
    onConnexion({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

par :

```
    connecter({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

Le compte.

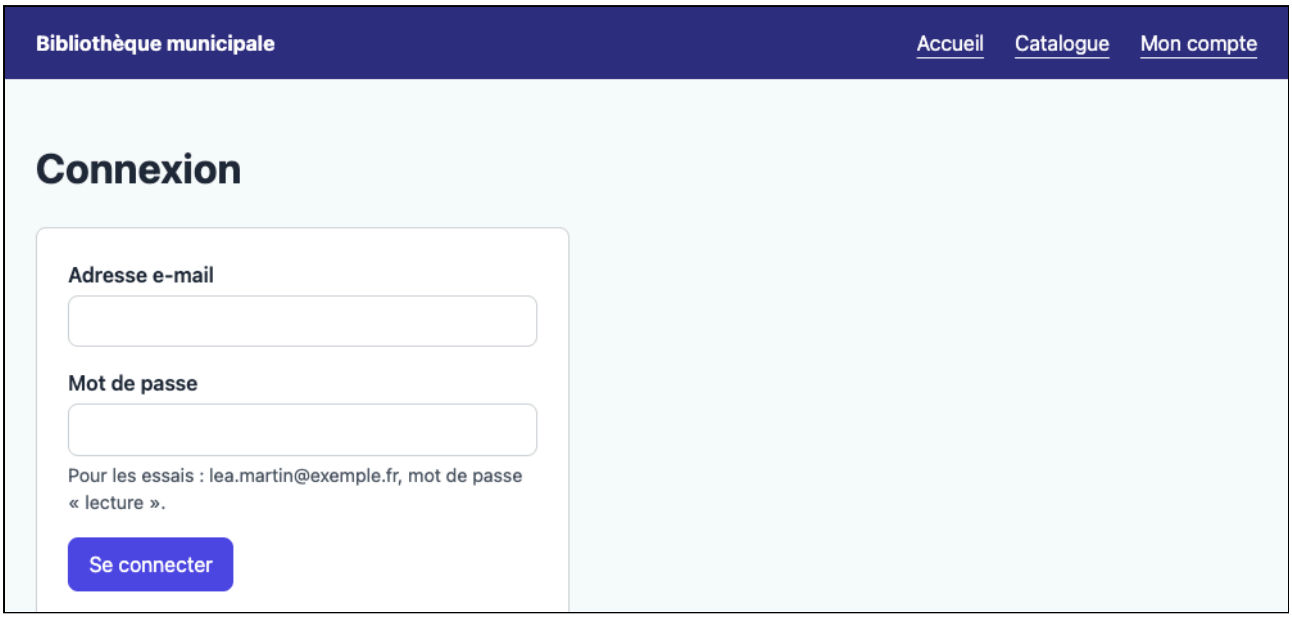
Remplacez tout le contenu de `src/pages/Compte.jsx` par :

```
import { useAuth } from '../contexts/AuthContext.jsx';

export default function Compte() {
  const { utilisateur } = useAuth();

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>Votre liste d'emprunts apparaîtra ici.</p>
    </>
  );
}
```

Vous devez voir : aucune erreur dans la console, et « Mon compte » vous renvoie à la connexion.

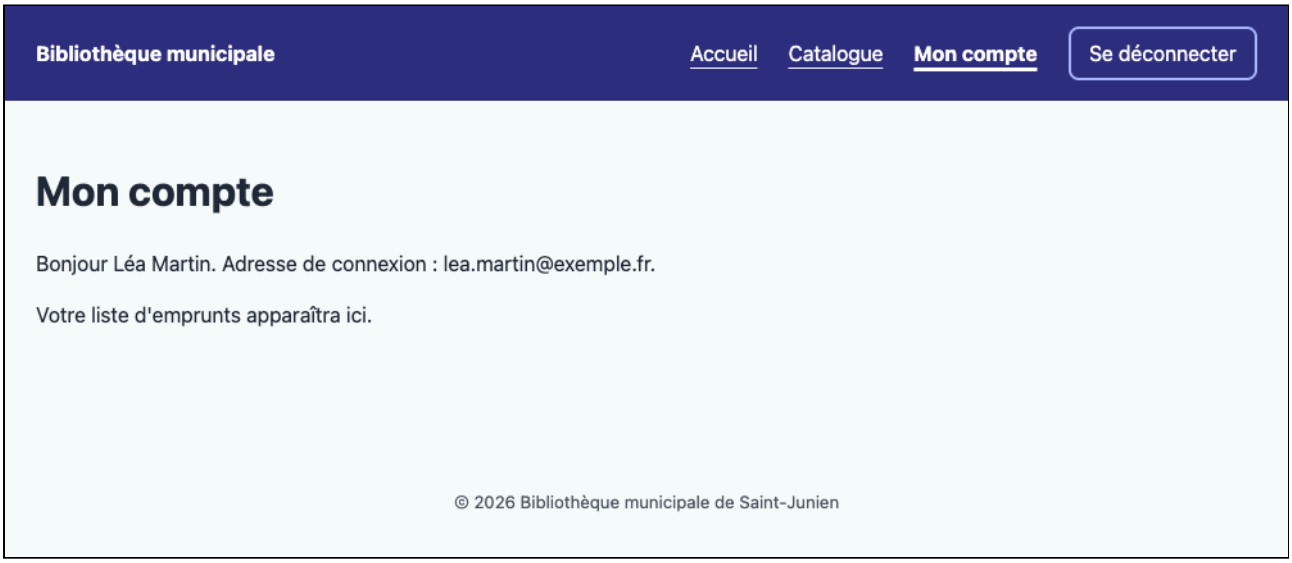


BIB-2, avec le bouchon : personne n'est connecté, « Mon compte » renvoie ici

Livrez le ticket, puis relisez BIB-1.

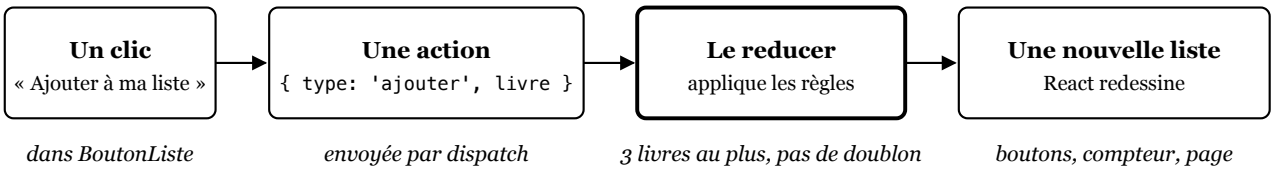
Ensemble, après les deux fusions

`git pull`, puis connectez-vous : `lea.martin@exemple.fr`, mot de passe `lecture`. La connexion marche, et « Mon compte » s'ouvre.



Fin du sprint 1 : Léa est connectée, le menu propose « Se déconnecter »

Sprint 2 · La liste



Ce que fait le reducer du sprint 2, à chaque clic

Le contrat : `useListe()` renvoie `{ liste, pleine, contient, ajouter, retirer, vider }`, et `MAXIMUM` vient de `listeReducer.js`.

BIB-3 · Le reducer et le contexte de la liste

Branche : `bib-3-reducer-liste`. Message : `BIB-3 : le reducer et le contexte de la liste`.

Remplacez tout le contenu de `src/contexts/listeReducer.js` par :

```
// La liste d'emprunts : au plus 3 livres, chacun une seule fois.
export const MAXIMUM = 3;

// Un reducer : (état actuel, action) → nouvel état. Il ne modifie rien, il ne fait
rien d'autre.
export function listeReducer(liste, action) {
  switch (action.type) {
    case 'ajouter': {
      const dejaLa = liste.some((l) => l.id === action.livre.id);
      if (dejaLa || liste.length >= MAXIMUM) {
        return liste;
      }
      const { id, titre, auteur } = action.livre;
      return [...liste, { id, titre, auteur }];
    }
    case 'retirer':
      return liste.filter((l) => l.id !== action.id);
    case 'vider':
      return [];
    default:
      throw new Error(`Action inconnue : ${action.type}`);
  }
}
```

Remplacez tout le contenu de `src/contexts/ListeContext.jsx` par :

```
import { createContext, useContext, useReducer } from 'react';
import { listeReducer, MAXIMUM } from './listeReducer.js';

const ListeContext = createContext(null);

export function ListeProvider({ children }) {
  const [liste, dispatch] = useReducer(listeReducer, []);

  const valeur = {
    liste,
    pleine: liste.length >= MAXIMUM,
    contient: (id) => liste.some((l) => l.id === id),
    ajouter: (livre) => dispatch({ type: 'ajouter', livre }),
    retirer: (id) => dispatch({ type: 'retirer', id }),
    vider: () => dispatch({ type: 'vider' }),
  };

  return <ListeContext value={valeur}>{children}</ListeContext>;
}

export function useListe() {
  const contexte = useContext(ListeContext);
  if (!contexte) {
    throw new Error('useListe doit être utilisé dans un <ListeProvider>.');
  }
  return contexte;
}
```

Dans `src/main.jsx`, juste après la ligne `import { AuthProvider } from './contexts/AuthContext.jsx';` :

```
import { ListeProvider } from './contexts/ListeContext.jsx';
```

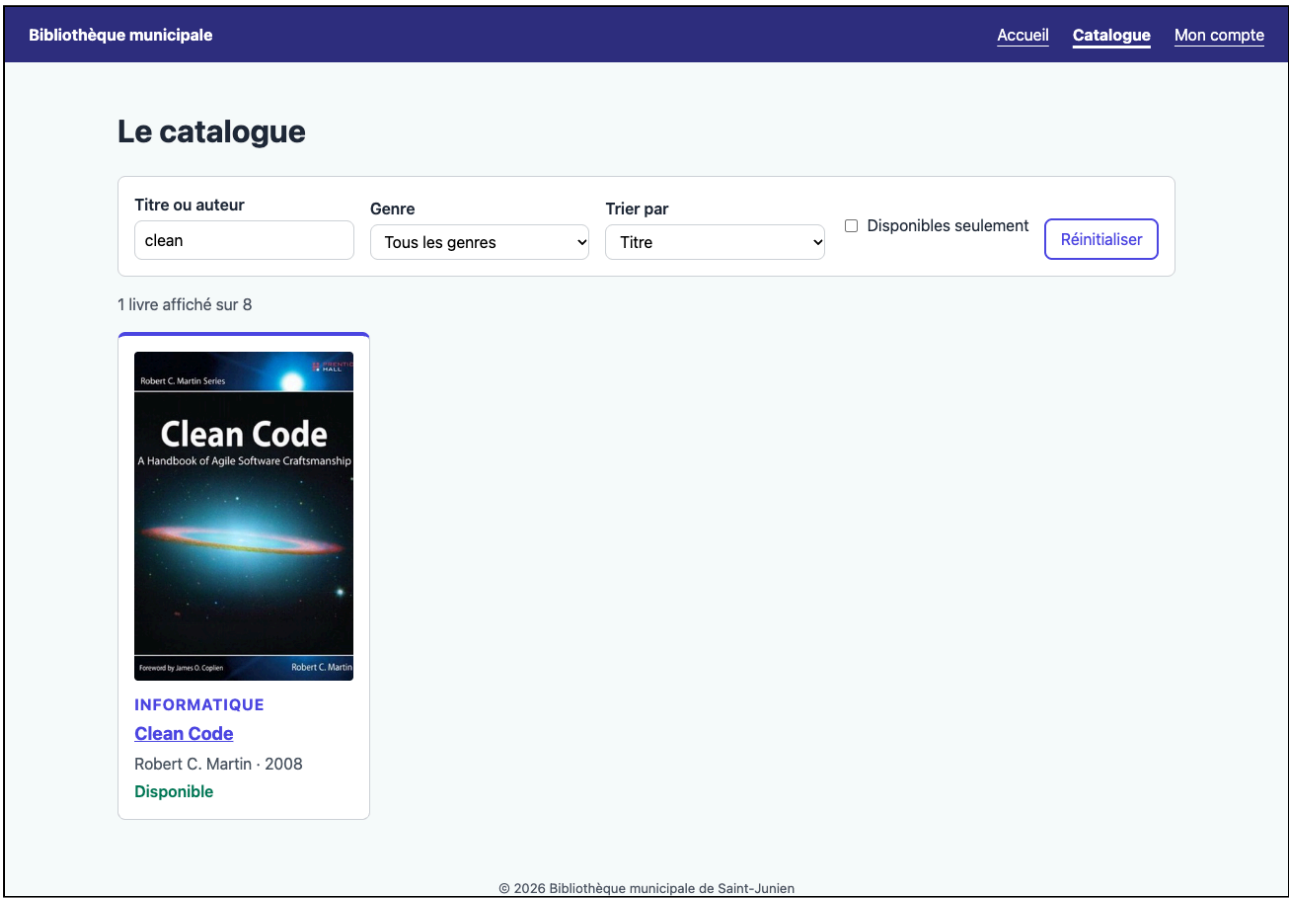
Dans `src/main.jsx`, remplacez :

```
        <App />
```

par :

```
        <ListeProvider>
          <App />
        </ListeProvider>
```

Vous devez voir : un `ListeProvider` sous l'`AuthProvider`, dans l'onglet *Components*. À l'écran, pas encore de bouton : c'est A qui les écrit.

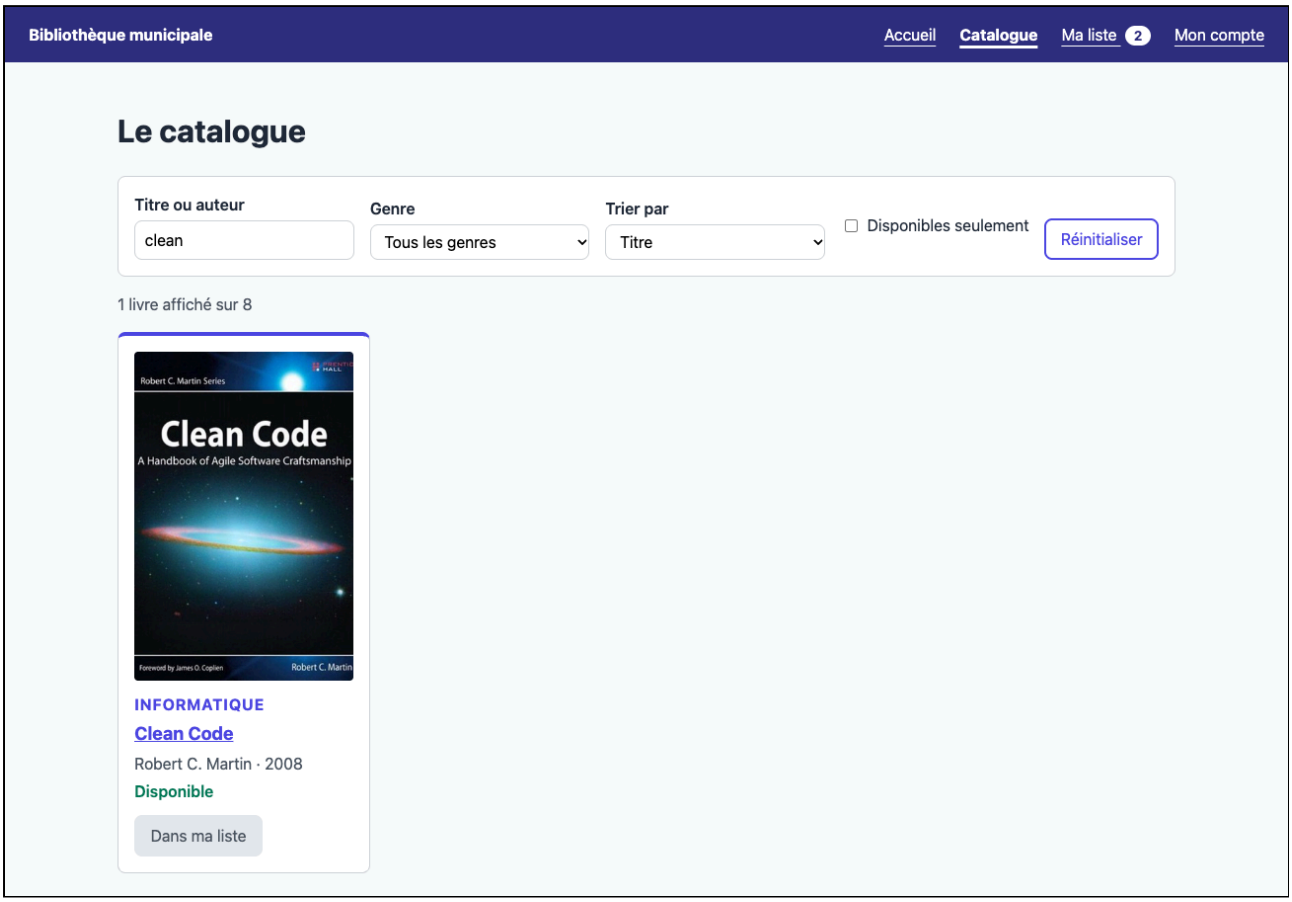


BIB-3 : le catalogue n'a pas encore de bouton

Livrez le ticket, puis relisez BIB-4.

Ensemble, après les deux fusions

`git pull`, puis ajoutez deux livres : le compteur du menu passe à 2. Dans « Ma liste », retirez-en un, puis videz la liste.



Fin du sprint 2 : deux livres ajoutés, le compteur dit 2

BIB-5 · Le bouton sur la fiche

Branche : `bib-5-bouton-fiche`. Message : `BIB-5 : le bouton sur la fiche`.

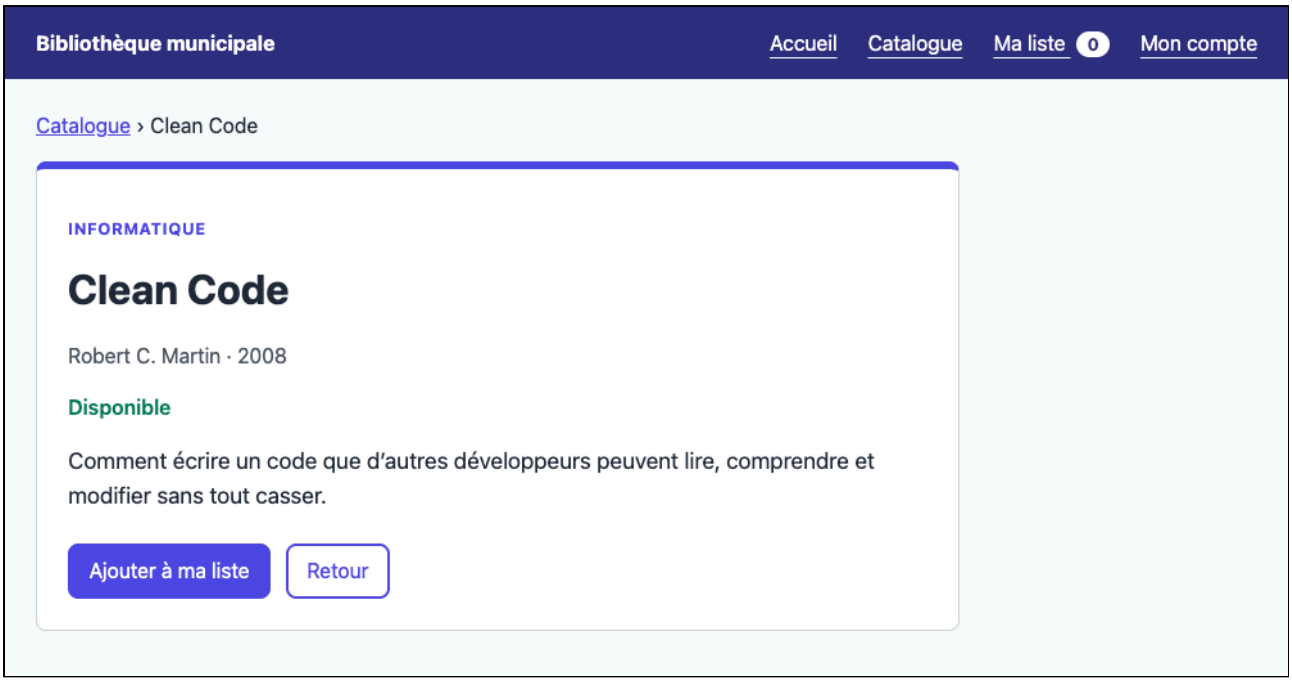
Dans `src/pages/FicheLivre.jsx`, juste après la ligne `import { livres } from '../data/livres.js';` :

```
import BoutonListe from '../components/BoutonListe.jsx';
```

Dans `src/pages/FicheLivre.jsx`, juste après la ligne `<div className="actions">` :

```
          <BoutonListe livre={livre} />
```

Vous devez voir : sur la fiche de *Clean Code*, le même bouton que sur sa carte.



BIB-5 : le bouton sur la fiche d'un livre

Livrez le ticket. A le relit.

Sprint 3 · La mémoire

BIB-7 · Le compte affiche la liste

Branche : `bib-7-compte`. Message : `BIB-7 : le compte affiche la liste`.

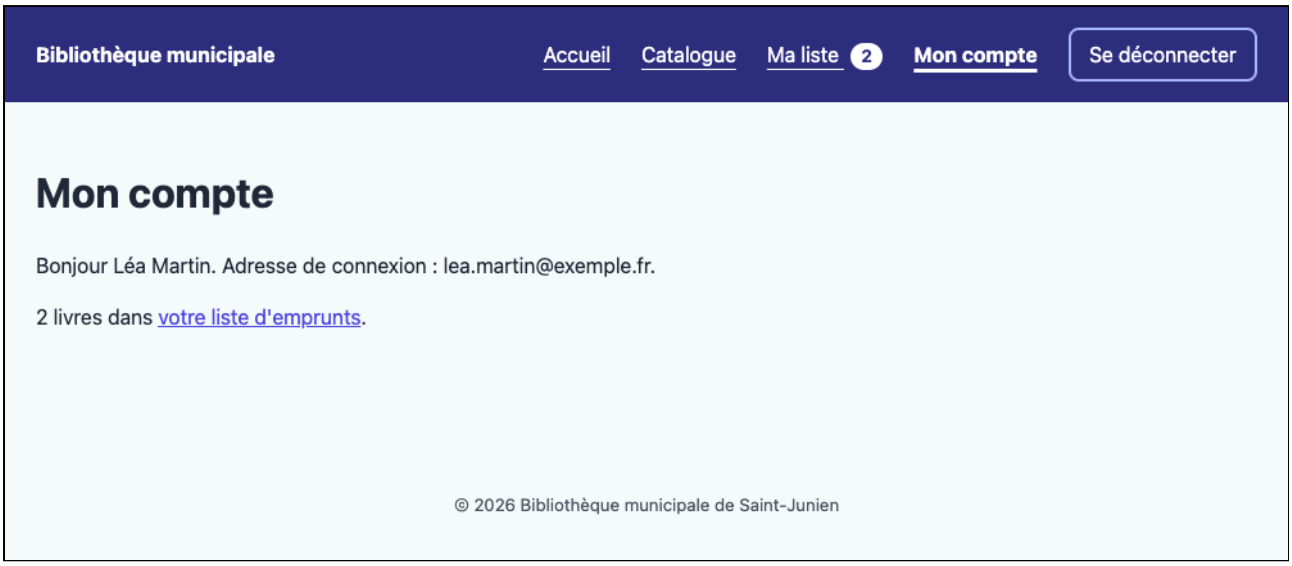
Remplacez tout le contenu de `src/pages/Compte.jsx` par :

```
import { Link } from 'react-router';
import { useAuth } from '../contexts/AuthContext.jsx';
import { useListe } from '../contexts/ListeContext.jsx';

export default function Compte() {
  const { utilisateur } = useAuth();
  const { liste } = useListe();
  const s = liste.length > 1 ? 's' : '';

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>
        {liste.length} livre{s} dans <Link to="/ma-liste">votre liste
d'emprunts</Link>.
      </p>
    </>
  );
}
```

Vous devez voir : connecté, avec deux livres, « 2 livres dans votre liste d'emprunts ».



BIB-7 : le compte rappelle le nombre de livres

Livrez le ticket, puis relisez BIB-6.

Ensemble, après les deux fusions

`git pull`. La liste survit à `F5`, et le compte l'affiche.

Rendre

Sur la page du dépôt GitHub : *Code*, puis *Download ZIP*. Renommez le fichier en `NOM1-NOM2-React-S3.zip`, et déposez-le une seule fois dans **Rendus — à déposer ici → Frameworks React → FM04 · Séance 3**.
